

Chapter 6

Computer Architecture

Part I

Textbook and References

Textbook:

Digital Design and Computer Architecture, 2nd Edition, David M.H and Sarah L.H, Morgan Kaufmann, Elsevier, 2013
(ISBN: 978-0-12-394424-5)

Reference(s):

1. Computer Architecture and Implementation, Harvey G. Cragon, Cambridge University Press, 2000 (ISBN: 0-52-165168-9)
2. Computer Architecture: A Quantitative Approach 4th Edition, David A. Patterson and John L. Hennessy, Morgan Kaufmann Publishers, 2005. (ISBN: 0-12-370490-1)

Simulator:

Tasm (Turbo Assembler)

Link: https://en.wikipedia.org/wiki/MIPS_instruction_set

Objective(s)

- One of the objectives of this section is to make each student to familiar with ISA (Instruction Set Architecture) -Machine Language of a CPU.
- Once the students gain the knowledge of Machine Language (ISA) of a CPU, then it is possible for them to design and simulate the ISA for a given CPU size (either a 16-bit CPU or 32-bit CPU or any) by using any conventional programming language .
- Finally, the students will get the knowledge of the complete operation of a computer.

Instructions – An Introduction

- The first step in understanding any computer architecture is to learn its **language**. The words in a computer's language are called **instructions**.
- The **computer's vocabulary** is called the ***instruction set***.
- **All programs running on a computer use the same instruction set.**
 - Even complex software applications, such as word processing and spreadsheet applications, are eventually compiled into a series of simple instructions such as ***add***, ***subtract***, ***jump***, etc.
- Computer instructions indicate both the *operation* (**opcode**) to perform and the *operation addresses* (**operands**) to use.
 - The operands may come from *memory*, from *registers*, or from the *instruction* itself.

Instructions – An Introduction

- Computer *hardware* understands only 1's and 0's, so **instructions are encoded as binary numbers** in a format called ***machine language***.
 - Just as we use letters to encode human language, **computers use binary numbers to encode machine language.**
- **Microprocessors** (means **CPUs**) are digital systems that read and execute ***machine language instructions***.
- However, humans consider reading ***machine language*** to be tedious, so we prefer to represent the **instructions** in a symbolic format called ***assembly language***.
 - ***Assembly language is more readable than machine language.***

Instructions – An Introduction

- The ***instruction sets*** of different architectures are more like different dialects than different languages.
- Almost all architectures (such as **Intel**) define basic instructions, such as *add*, *subtract*, and *jump*, that operate on **memory** or **registers**.

Assembly Language

- **Assembly language** is the human-readable representation of the computer's **Machine Language**
 - Assembly language is an *encoded instruction set* of a computer.
- Each assembly language instruction specifies both the *operation code* to perform and the *operands* on which to operate.

Instructions

- The most common operation computers perform is *addition*. Code **Example 6.1** shows code for adding variables ***b*** and ***c*** and writing the result to ***a***.
- The code is shown on the left in a **high-level language** (using the syntax of C, C++, and Java), and then rewritten on the right in **MIPS assembly language**.
 - **MIPS** (*Microprocessor- without Interlocked Pipeline Stage*) processor

Code Example 6.1 ADDITION

High-Level Code	MIPS Assembly Code
<code>a = b + c;</code>	<code>add a, b, c</code>

Code Example 6.2 SUBTRACTION

High-Level Code	MIPS Assembly Code
<code>a = b - c;</code>	<code>sub a, b, c</code>

Instructions

- The first part of the assembly instruction, ***add***, is called the ***mnemonic (op-code)*** and indicates what operation to perform.
- The operation is performed on ***b*** and ***c***, the ***source operands***, and the result is written to ***a***, the ***destination operand***.
- In this introduces the **MIPS** (***Microprocessor- without Interlocked Pipeline Stage***) **processor** architecture that was first developed by **John Hennessy** and his colleagues at Stanford in the 1980s.
 - https://en.wikipedia.org/wiki/MIPS_instruction_set
 - **MIPS processors** are used by, among others, Silicon Graphics, Nintendo, and Cisco

RISC and CISC Machines

- The **MIPS instruction set** makes the common case fast by including only simple, commonly used instructions.
- *The number of instructions is kept small so that the hardware required to decode the instruction and its operands can be simple, small, and fast.*
- More elaborate operations that are less common are performed using sequences of multiple simple instructions. Thus, **MIPS is a reduced instruction set computer (RISC)** architecture.
- Architectures with many complex instructions, such as **Intel's x86** architecture, are **complex instruction set computers (CISC)**.

RISC and CISC Machines

- A **RISC** architecture minimizes the hardware complexity and the necessary instruction encoding by keeping the set of distinct instructions small.
 - For example, *an instruction set with 64 simple instructions would need $\log_2 64 = 6$ bits to encode the operation.*
 - *An instruction set with 256 complex instructions would need $\log_2 256 = 8$ bits of encoding per instruction.*
- In a **CISC machine**, even though the complex instructions may be used only rarely, they add overhead to all instructions, even the simple ones.

Operands: Registers, Memory, and Constants

- An instruction operates on **operands** (Operation addresses).
- In Code **Example 6.1** the variables *a*, *b*, and *c* are all **operands**.
- **But computers operate on 1's and 0's, not variable names.**
- The instructions need a physical location from which to retrieve the binary data.
- **Operands** can be stored in *registers* or *memory*, or they may be **constants** stored in the instruction itself.

Operands: Registers, Memory, and Constants

- Computers use various locations to hold **operands** in order to optimize for speed and data capacity.
- **Operands** stored as *constants* or in *registers* are accessed quickly inside a CPU (**why?**), but they hold only a small amount of data.
 - Additional data must be accessed from memory, which is large but slow.
- **MIPS-32** is called a **32-bit architecture** because it operates on 32-bit data (currently, the MIPS architecture has been extended to 64 bits in commercial products).

Registers

- Instructions need to access operands quickly so that they can run fast.
- But **operands** stored in memory take a long time to retrieve.
- Therefore, most architectures specify a few set of **registers (called GPRs)** that hold commonly used operands.
- The MIPS architecture uses **32 registers**, called **the register set** or **register file**.
- **The fewer the registers**, the faster they can be accessed.
- This leads to the design principle: *Smaller is faster*.

Registers

- **Code Example 6.4** shows the **add instruction** with **register operands**.
- MIPS **register names** are preceded by the **\$** sign.
- The variables ***a***, ***b***, and ***c*** are arbitrarily placed in **\$s0**, **\$s1**, and **\$s2**.
 - The name **\$s1** is pronounced “register s1” or “dollar s1”. The instruction adds the 32-bit values contained in **\$s1** (b) and **\$s2** (c) and writes the 32-bit result to **\$s0** (a).

Registers

Code Example 6.4 REGISTER OPERANDS

High-Level Code	MIPS Assembly Code
<code>a = b + c;</code>	<code># \$s0 = a, \$s1 = b, \$s2 = c</code> <code>add \$s0, \$s1, \$s2 # a = b + c</code>

Code Example 6.5 TEMPORARY REGISTERS

High-Level Code	MIPS Assembly Code
<code>a = b + c - d;</code>	<code># \$s0 = a, \$s1 = b, \$s2 = c, \$s3 = d</code> <code>sub \$t0, \$s2, \$s3 # t = c - d</code> <code>add \$s0, \$s1, \$t0 # a = b + t</code>

Registers

- **MIPS generally stores variables in 18 of the 32 registers:** \$s0–\$s7, and \$t0–\$t9.
- Register names beginning with **\$s** are called **saved registers**. Following MIPS convention, these registers store variables such as ***a***, ***b***, and ***c***.
- **Saved registers** have special connotations when they are used with **function calls**.
- Register names beginning with **\$t** are called **temporary registers**. They are used for storing **temporary variables**.
 - **Code Example 6.5** shows MIPS assembly code using a temporary register, **\$t0**, to store the intermediate calculation of ***c* – *d***.

Registers

- The MIPS architecture defines **32 registers**.
- Each register has a name and a number ranging from **0** to **31**.
- **Table 6.1** lists the name, number, and use for each register. Register 0 (\$0) always contains the value 0 because this constant is so frequently used in computer programs.

Registers

Table 6.1 MIPS register set

Name	Number	Use
\$0	0	the constant value 0
\$at	1	assembler temporary
\$v0-\$v1	2-3	function return value
\$a0-\$a3	4-7	function arguments
\$t0-\$t7	8-15	temporary variables
\$s0-\$s7	16-23	saved variables
\$t8-\$t9	24-25	temporary variables
\$k0-\$k1	26-27	operating system (OS) temporaries
\$gp	28	global pointer
\$sp	29	stack pointer
\$fp	30	frame pointer
\$ra	31	function return address

MIPS Memory

- If **registers** were the only storage space for **operands**, we would be confined (limited) to simple programs with no more than 32 variables.
- However, data can also be stored in **memory**.
- When compared to the **register file**, **memory** has many data locations, but accessing it takes a longer amount of time.
- Whereas **the register file is small and fast**, **memory is large and slow**. For this reason, commonly used variables are kept in registers.
 - By using a **combination of memory and registers**, a program can access a large amount of data fairly quickly.

MIPS Memory

- The MIPS architecture uses **32-bit memory addresses** and **32-bit data words**.
- MIPS uses a **byte-addressable memory**. That is, each byte in memory has a 32-bit unique address.
- However, for explanation purposes only, we first introduce a **word-addressable memory**, and afterward describe the **MIPS's byte-addressable memory**.
- **Figure 6.1** shows a memory array that is **word-addressable**.
 - It means that *each 32-bit data word has a unique 32-bit address*.
 - Both the 32-bit word address and the 32-bit data value are written in hexadecimal in **Figure 6.1**.

MIPS Memory

Word Address	Data	
⋮	⋮	⋮
00000003	4 0 F 3 0 7 8 8	Word 3
00000002	0 1 E E 2 8 4 2	Word 2
00000001	F 2 F 1 A C 0 7	Word 1
00000000	A B C D E F 7 8	Word 0

Figure 6.1 Word-addressable memory

MIPS Memory

- MIPS uses the load word instruction (lw), to read a data word (32-bit) from memory into a register. Code **Example 6.6** loads memory word 1 into **\$s3**.
- The **lw** instruction specifies the effective address in memory as the sum of a base address and an offset.
 - The **base address is a register** (written in parentheses in the instruction, see the code **example 6.6**). The **offset** is a **constant** (written before the parentheses).
- In **Code Example 6.6**, the **base address** is register **\$0**, which holds the value **0**, and the **offset** is **1**, so the **lw** instruction reads from **memory address $(\$0 + 1) = 1$** .
 - The **effective address** or **physical address** of the memory is 1

MIPS Memory

Code Example 6.6 READING WORD-ADDRESSABLE MEMORY

Assembly Code

```
# This assembly code (unlike MIPS) assumes word-addressable memory  
lw $s3, 1($0)    # read memory word 1 into $s3
```


MIPS Memory

- The MIPS memory model, however, is **byte-addressable**, and is **not word-addressable** as mentioned before.
- Each **data byte** has a unique address.
- A **32-bit word** consists of **four 8-bit bytes**. So each word address is a multiple of 4, as shown in **Figure 6.2**.
- Again, both the 32-bit word address and the data value are given in **hexadecimal**.
 - **Code Example 6.8** shows how to read and write words in the MIPS **byte-addressable memory**. The word address is four times the word number.
 - The MIPS assembly code reads words 0, 2, and 3 and writes words 1, 8, and 100. The offset can be written in decimal or hexadecimal.

MIPS Memory

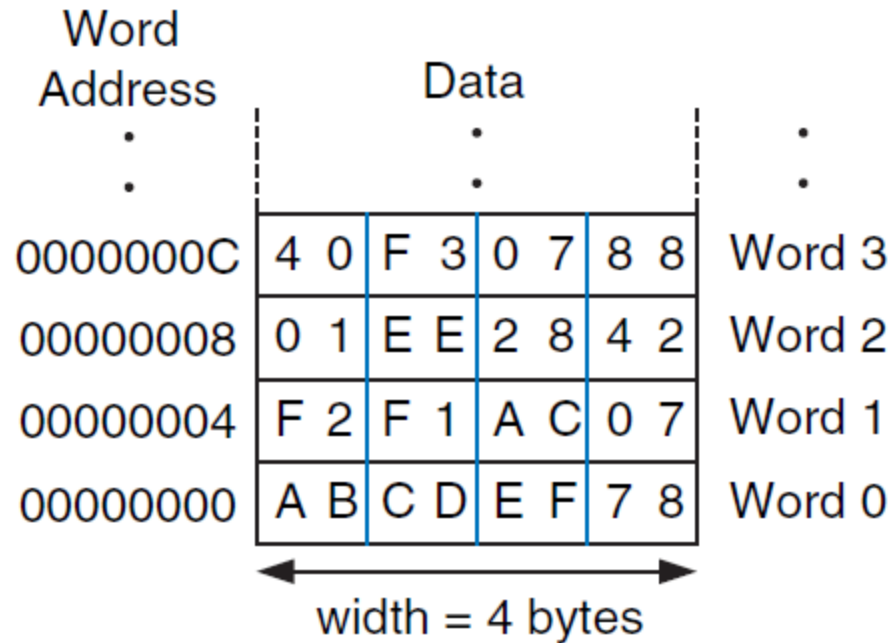


Figure 6.2 Byte-addressable memory

MIPS Memory

Code Example 6.8 ACCESSING BYTE-ADDRESSABLE MEMORY

MIPS Assembly Code

```
lw $s1, 8($0)      # read data word 2 (0x01EE2842) into $s1
lw $s2, 0xC($0)     # read data word 3 (0x40F30788) into $s2
sw $s3, 4($0)       # write $s3 to data word 1
sw $s4, 0x20($0)    # write $s4 to data word 8
sw $s5, 400($0)     # write $s5 to data word 100
```

MIPS Memory

- **Byte-addressable memories** are organized in a **big-endian** or **little-endian** fashion, as shown in **Figure 6.3**.
- In **big-endian machines**, bytes are numbered starting with 0 at the big (most significant) end.
 - Store the most significant byte (MSB) in the smallest address of memory.
- In **little-endian machines**, bytes are numbered starting with 0 at the little (least significant) end. Example: Intel
 - Store the *least* significant byte (LSB) in the smallest address of memory
- Word addresses are the same in both formats and refer to the same four bytes. Only the addresses of bytes within a word differ.

MIPS Memory

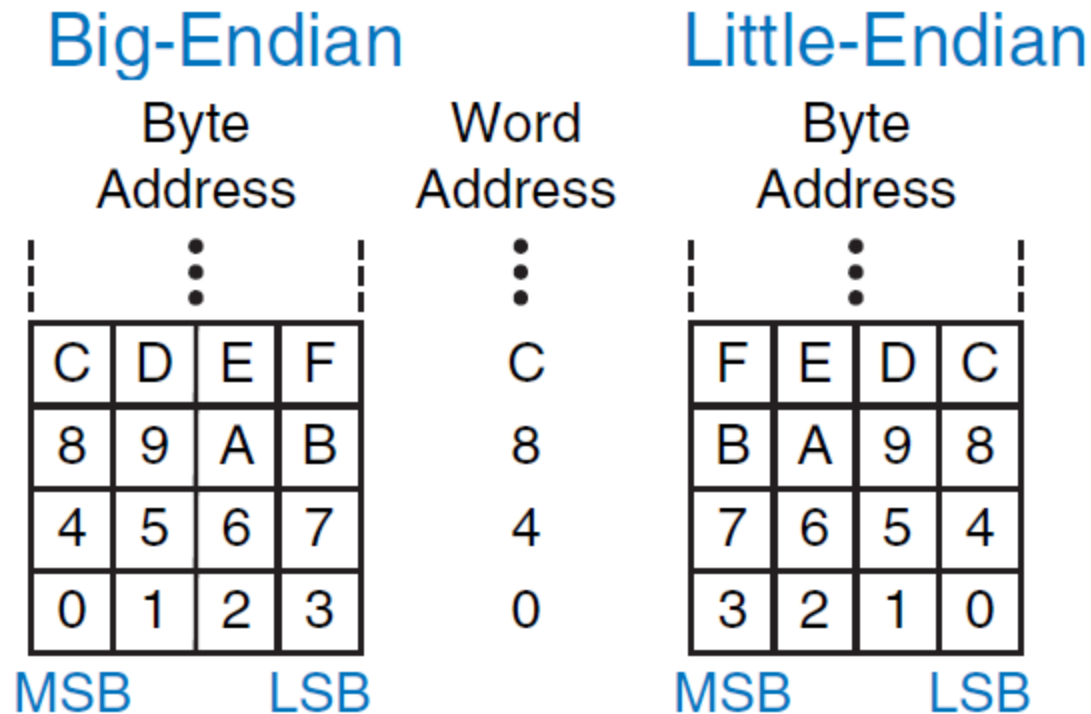


Figure 6.3 Big- and little-endian memory addressing

MIPS Memory

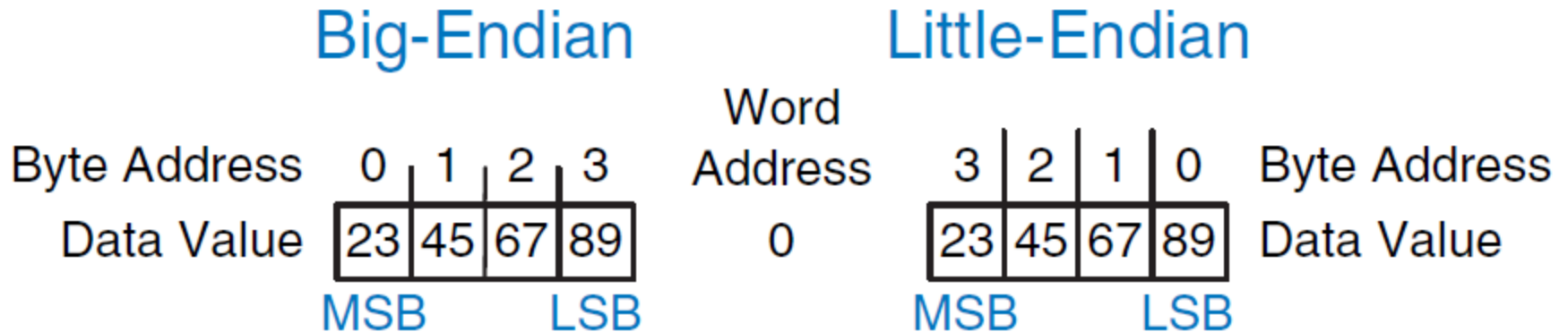


Figure 6.4 Big-endian and little-endian data storage

Constants/Immediates

- **Load word** and **store word**, **lw** and **sw**, also illustrate the use of **constants** in MIPS instructions.
- These constants are called **immediates**, because *their values are immediately available from the instruction and do not require a register or memory access.*
- **Add immediate**, **addi**, is another common MIPS instruction that uses an **immediate operand**.
- **addi** adds the **immediate** specified in the instruction to a value in a register, as shown in **Code Example 6.9**.

Constants/Immediates

Code Example 6.9 IMMEDIATE OPERANDS

High-Level Code

```
a = a + 4;  
b = a - 12;
```

MIPS Assembly Code

```
# $s0 = a, $s1 = b  
addi $s0, $s0, 4          # a = a + 4  
addi $s1, $s0, -12        # b = a - 12
```


MIPS Machine Language

- A program written in **Assembly language** is translated from mnemonics to a representation using only 1's and 0's called ***machine language***.
- **MIPS** uses 32-bit instructions.
 - Again, simplicity favors regularity, and the most regular choice is to **encode all instructions as words** that can be stored in memory.
- Even though some instructions may not require all 32 bits of encoding, ***variable-length instructions*** would add too much complexity.

MIPS Machine Language

- MIPS makes the compromise of defining three instruction formats: **R-type**, **I-type**, and **J-type**.
 - This small number of formats allows for some regularity among all the types, and thus simpler hardware, while also accommodating different instruction needs, such as the need to encode large constants in the instruction.
- **R-type instructions** operate on **three registers**.
- **I-type instructions** operate on **two registers** and a **16-bit immediate**.
- **J-type (jump) instructions** operate on one 26-bit immediate (**memory offset address**).

Code Example 6.12 CONDITIONAL BRANCHING USING beq

MIPS Assembly Code

```
addi $s0, $0, 4      # $s0 = 0 + 4 = 4
addi $s1, $0, 1      # $s1 = 0 + 1 = 1
sll  $s1, $s1, 2      # $s1 = 1 << 2 = 4
beq  $s0, $s1, target # $s0 == $s1, so branch is taken
addi $s1, $s1, 1      # not executed
sub  $s1, $s1, $s0     # not executed

target:
add  $s1, $s1, $s0     # $s1 = 4 + 4 = 8
```

R-Type Instructions

- The name **R-type** is short for **register-type**.
- **R-type instructions** use **three registers as operands**: *two as sources*, and *one as a destination*.
- **Figure 6.5** shows the **R-type machine instruction format**.
- The **32-bit instruction** has **six fields**: *op*, *rs*, *rt*, *rd*, *shamt*, and *funct*. Each field is five or six bits, as indicated.
- The operation the instruction performs is encoded in the two fields highlighted in blue: **op** (also called **opcode** or **operation code**) and **funct** (also called the **function**).

R-Type Instructions

- All **R-type instructions** have an **opcode** of **0**. The specific **R-type** operation is determined by the **funct** field.
 - For example, the *opcode* and *funct* fields for the **add instruction** are 0 and 32 , respectively. Similarly, the sub instruction has an *opcode* and *funct* field of 0 and 34.

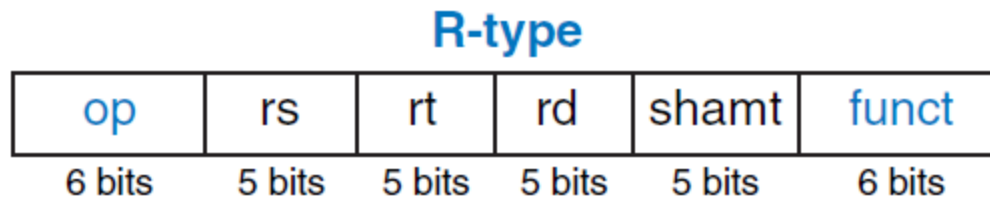


Figure 6.5 R-type machine instruction format

R-Type Instructions

- The operands are encoded in **the three fields**:
 - **rs**, **rt**, and **rd**. The first two registers, **rs** and **rt**, are the source registers; **rd** is the destination register.
 - The fields contain the register numbers that were given in **Table 6.1**. For example, **\$s0** is register 16.
- The fifth field, ***shamt***, is used only in ***shift operations***.
 - In those instructions, the binary value stored in the 5-bit *shamt* field indicates the amount to **shift**.
 - For all other **R-type** instructions, *shamt* is 0.

R-Type Instructions

- **Figure 6.6** shows the **machine code (ISA)** for the **R-type instructions** *add* and *sub*.
 - Notice that the *destination* is the first register in an assembly language instruction, but it is the third register field (***rd***) in the machine language instruction.
 - For example, the assembly instruction `add $s0, $s1, $s2` has ***rs*** = \$s1 (17), ***rt*** = \$s2 (18), and ***rd*** = \$s0 (16).

R-Type Instructions

Assembly Code

add \$s0, \$s1, \$s2

sub \$t0, \$t3, \$t5

Field Values

op	rs	rt	rd	shamt	funct
0	17	18	16	0	32
0	11	13	8	0	34
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Machine Code

op	rs	rt	rd	shamt	funct	
000000	10001	10010	10000	00000	100000	(0x02328020)
000000	01011	01101	01000	00000	100010	(0x016D4022)
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	

Figure 6.6 Machine code for R-type instructions

R-Type Instructions

Example 6.3 TRANSLATING ASSEMBLY LANGUAGE TO MACHINE LANGUAGE

Translate the following assembly language statement into machine language.

```
add $t0, $s4, $s5
```

Solution: According to Table 6.1, \$t0, \$s4, and \$s5 are registers 8, 20, and 21. According to Tables B.1 and B.2, add has an opcode of 0 and a funct code of 32. Thus, the fields and machine code are given in Figure 6.7. The easiest way to write the machine language in hexadecimal is to first write it in binary, then look at consecutive groups of four bits, which correspond to hexadecimal digits (indicated in blue). Hence, the machine language instruction is 0x02954020.

R-Type Instructions

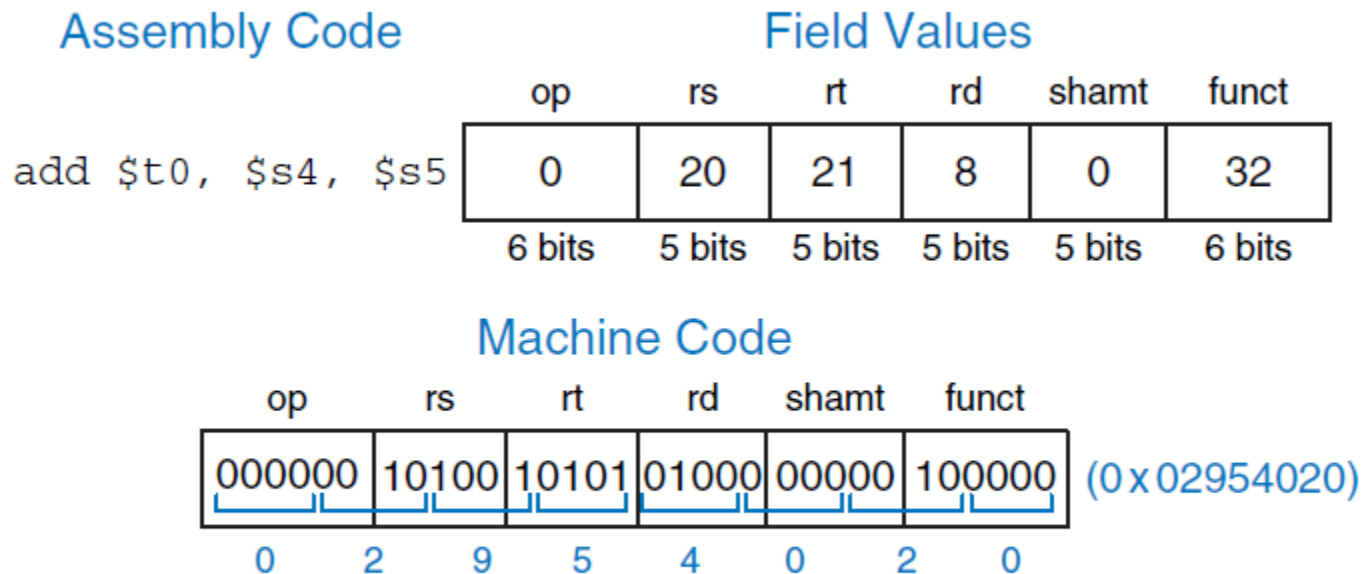


Figure 6.7 Machine code for the R-type instruction of Example 6.3

I-Type Instructions

- The name **I-type** is short for **immediate-type**.
- **I-type instructions** use two register operands and one 16-bit immediate operand. **Figure 6.8** shows the **I-type** machine instruction format.
- The 32-bit instruction has **four** fields: **op**, **rs**, **rt**, and **imm**. The *first three fields*, **op**, **rs**, and **rt**, are like those of **R-type** instructions. The **imm** field holds the **16-bit immediate**.

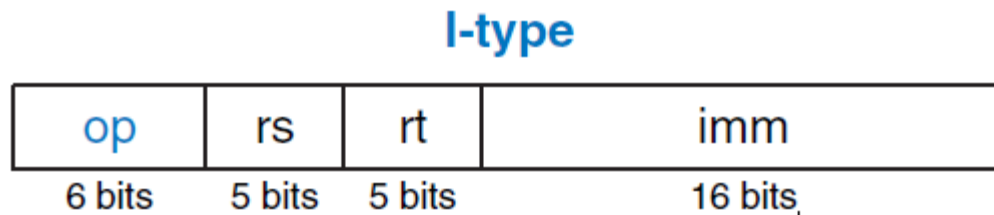


Figure 6.8 I-type instruction format

I-Type Instructions

- The operation is determined by the **opcode**, highlighted in blue. The operands are specified in the three fields ***rs***, ***rt***, and ***imm***.
 - ***rs*** and ***imm*** are always used as source operands.
 - ***rt*** is used as a destination for some instructions (such as ***addi*** and ***lw***) but as another source for others (such as ***sw***).
- **Figure 6.9** shows several examples of **encoding I-type instructions**.
 - Recall that negative immediate values are represented using 16-bit two's complement notation.
 - ***rt*** is listed first in the assembly language instruction when it is used as a destination, but it is the second register field in the machine language instruction.

I-Type Instructions

Assembly Code	Field Values				Machine Code				
	op	rs	rt	imm	op	rs	rt	imm	
addi \$s0, \$s1, 5	8	17	16	5	001000	10001	10000	0000 0000 0000 0101	(0x22300005)
addi \$t0, \$s3, -12	8	19	8	-12	001000	10011	01000	1111 1111 1111 0100	(0x2268FFF4)
lw \$t2, 32(\$0)	35	0	10	32	100011	00000	01010	0000 0000 0010 0000	(0x8C0A0020)
sw \$s1, 4(\$t1)	43	9	17	4	101011	01001	10001	0000 0000 0000 0100	(0xAD310004)
	6 bits	5 bits	5 bits	16 bits	6 bits	5 bits	5 bits	16 bits	

Figure 6.9 Machine code for I-type instructions

I-Type Instructions

Example 6.4 TRANSLATING I-TYPE ASSEMBLY INSTRUCTIONS INTO MACHINE CODE

Translate the following I-type instruction into machine code.

```
lw $s3, -24($s4)
```

Solution: According to Table 6.1, \$s3 and \$s4 are registers 19 and 20, respectively. Table B.1 indicates that lw has an opcode of 35. rs specifies the base address, \$s4, and rt specifies the destination register, \$s3. The immediate, imm, encodes the 16-bit offset, -24. The fields and machine code are given in Figure 6.10.

I-Type Instructions

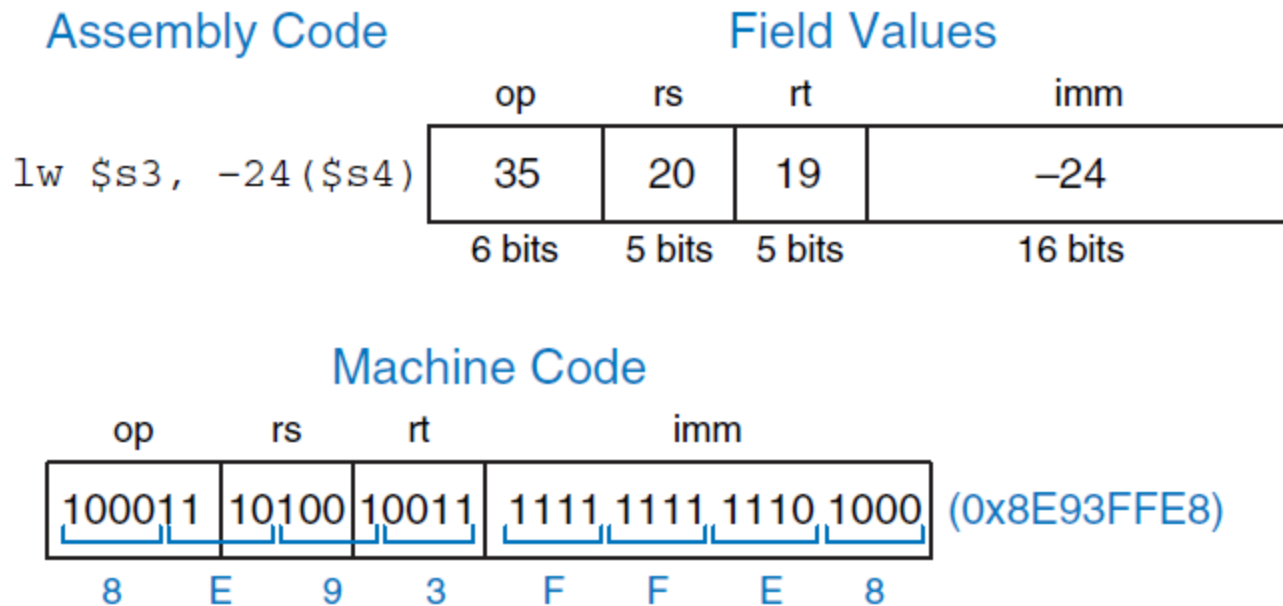


Figure 6.10 Machine code for an I-type instruction

I-Type Instructions

- **I-type instructions** have a **16-bit immediate field**, but the immediates are used in 32-bit operations.
 - For example, *lw* adds a **16-bit offset to a 32-bit base register**.
 - What should go in the upper half of the 32 bits?
 - For **positive immediates**, the upper half should be all 0's, but for **negative immediates**, the upper half should be all 1's.
 - An ***N*-bit two's complement number** is **sign-extended** to an ***M*-bit number** ($M > N$) by copying the **sign bit** (*most significant bit*) of the ***N*-bit number** into all of the *upper bits* of the ***M*-bit number**.
- **Sign-extending a two's complement number does not change its value.**

J-Type Instructions

- The name **J-type** is short for **jump-type**.
- This format is used only with **jump instructions**. This instruction format uses a single **26-bit address operand**, ***addr***, as shown in **Figure 6.11**.
 - Like other formats, **J-type instructions** begin with a **6-bit opcode** and the remaining bits are used to **specify an address**, ***addr*** to jump

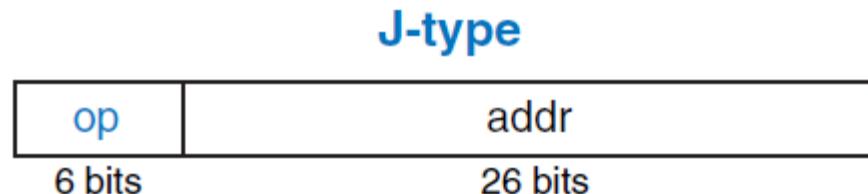


Figure 6.11 J-type instruction format

J-Type Instructions

Example 6.5 TRANSLATING MACHINE LANGUAGE TO ASSEMBLY LANGUAGE

Translate the following machine language code into assembly language.

0x2237FFF1

0x02F34022

Solution: First, we represent each instruction in binary and look at the six most significant bits to find the opcode for each instruction, as shown in [Figure 6.12](#). The opcode determines how to interpret the rest of the bits. The opcodes are 001000_2 (8_{10}) and 000000_2 (0_{10}), indicating an `addi` and R-type instruction, respectively. The `funct` field of the R-type instruction is 100010_2 (34_{10}), indicating that it is a `sub` instruction. [Figure 6.12](#) shows the assembly code equivalent of the two machine instructions.

J-Type Instructions

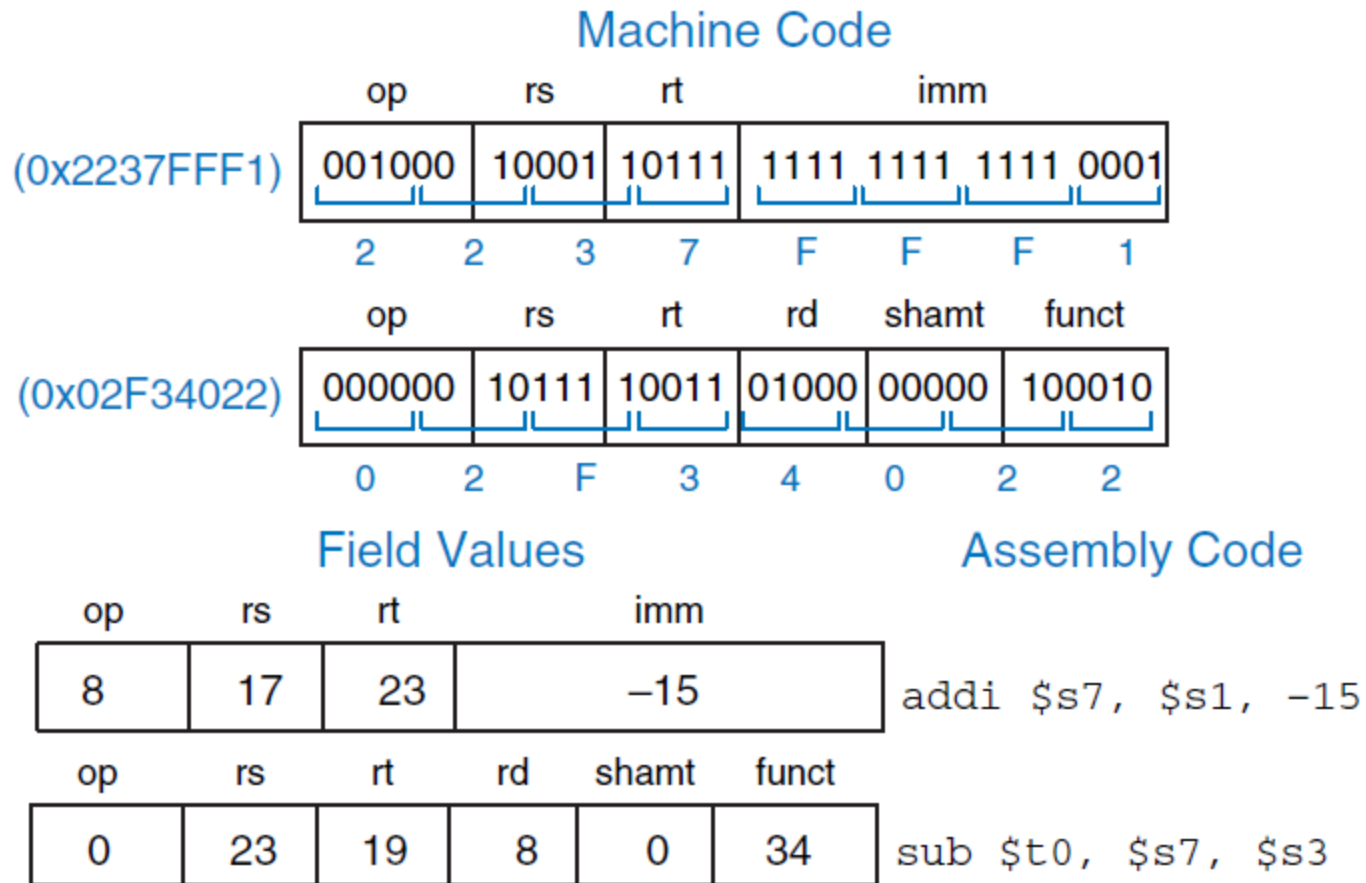


Figure 6.12 Machine code to assembly code translation

Interpreting Machine Language Code

- To interpret **machine language**, one must decipher (means *to discover the meaning of a complexwritten script*) the fields of each 32-bit instruction word.
- Different instructions use different formats, but all formats start with a 6-bit **opcode field**.
- Thus, the best place to begin is to look at the **opcode**. *If it is 0*, the instruction is **R-type**; otherwise it is **I-type** or **J-type**.

The Power of the Stored Program

- A **program** written in machine language is a series of 32-bit numbers representing the instructions.
- Like other binary numbers, these instructions can be stored in memory. This is called the **stored program concept**, and it is a key reason why computers are so powerful.
 - Running a different program does not require large amounts of time and effort to reconfigure or rewire hardware; it only requires writing the new program to memory.
 - Instead of dedicated hardware, the **stored program** offers general purpose computing.
 - In this way, a computer can execute applications simply by changing the stored program.

The Power of the Stored Program

- **Instructions** in a stored program are *retrieved*, or ***fetched***, from **memory** and **executed** by the processor.
- Even large, complex programs are simplified to *a series of memory reads and instruction executions.*
- **Figure 6.13** shows how machine instructions are stored in memory.
- In MIPS programs, the instructions are normally stored starting at memory address 0x00400000.
 - Remember that MIPS memory is **byte-addressable**, so 32-bit (4-byte) instruction addresses advance by 4 bytes.

The Power of the Stored Program

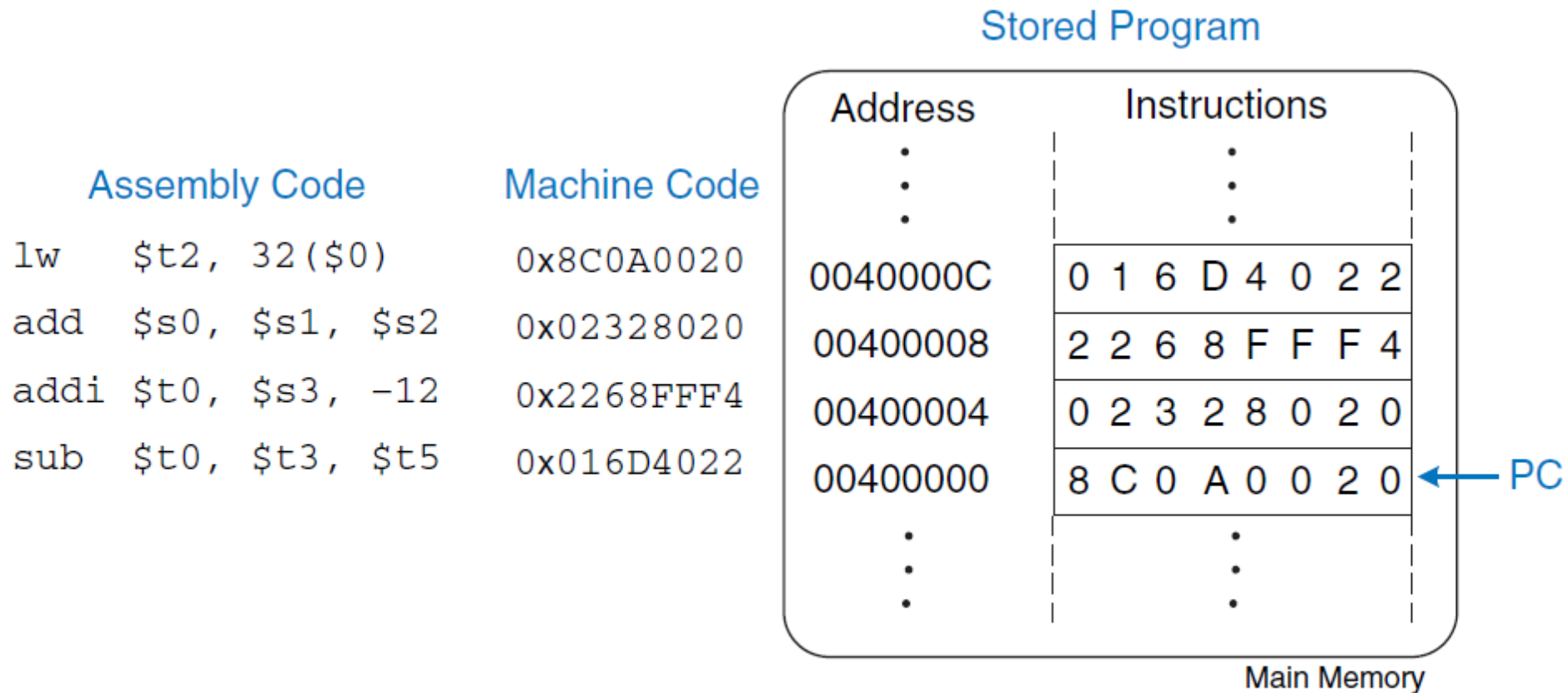


Figure 6.13 Stored program

The Power of the Stored Program

- To run or execute the stored program, the processor fetches (reads) the instructions from memory sequentially.
- The fetched instructions are then decoded and executed by the digital hardware.
- The address of the current instruction is kept in a **32-bit register** called the **program counter (PC)**.
 - To execute the code in **Figure 6.13**, the **operating system** sets the **PC** to address 0x00400000.
 - The processor reads the instruction at that memory address and executes the instruction, 0x8C0A0020 (see **Figure 6.13**).
 - The processor then increments the **PC** by 4 to 0x00400004, fetches and executes that instruction, and repeats.

The Power of the Stored Program

- The architectural state of a **microprocessor** (means **CPU**) holds the state of a program.
- For MIPS, the architectural state consists of the **register file** and **PC (program counter)**.
- If the **operating system** saves the architectural state at some point in the program, it can interrupt the program, do something else, then restore the state such that the program continues properly, unaware that it was ever interrupted.

Programming

- Software languages such as **C** or **Java** are called **high-level programming languages** because they are written at a more abstract level than assembly language.
- Many **high-level languages** use common software constructs such as arithmetic and logical operations, if/else statements, ***for*** and ***while*** loops, **array indexing**, and **function calls**.
 - See **Appendix C** for more examples of these constructs in C. In this section, we explore how to translate these high-level constructs into MIPS assembly code.

Arithmetic/Logical Instructions

Logical Instructions

- **MIPS logical operations** include *and*, *or*, *xor*, and *nor*.
- These **R-type** instructions operate **bit-by-bit** on two source registers and write the result to the destination register.
- **Figure 6.14** shows examples of these operations on the two source values 0xFFFF0000 and 0x46A1F0B7. The figure shows the values stored in the destination register, rd, after the instruction executes.

Arithmetic/Logical Instructions

Assembly Code

and \$s3, \$s1, \$s2

or $\$s4, \$s1, \$s2$

```
xor $s5, $s1, $s2
```

```
nor $s6, $s1, $s2
```

Source Registers

\$s1	1111	1111	1111	1111	0000	0000	0000	0000
------	------	------	------	------	------	------	------	------

\$s2	0100	0110	1010	0001	1111	0000	1011	0111
------	------	------	------	------	------	------	------	------

Result

\$s3	0100	0110	1010	0001	0000	0000	0000	0000
------	------	------	------	------	------	------	------	------

[illegible]

\$s5	1011	1001	0101	1110	1111	0000	1011	0111
------	------	------	------	------	------	------	------	------

[illegible]

Figure 6.14 Logical operations

Arithmetic/Logical Instructions

- The `and` instruction is useful for masking bits (i.e., forcing unwanted bits to 0). For example, in **Figure 6.14**, $0xFFFF0000 \text{ AND } 0x46A1F0B7 = 0x46A10000$.
- The ***and*** instruction masks off the bottom two bytes and places the unmasked top two bytes of `$s2`, `0x46A1`, in `$s3`.
- Any subset of register bits can be masked.
- The **`or`** instruction is useful for combining bits from two registers. For example, $0x347A0000 \text{ OR } 0x000072FC = 0x347A72FC$, combination of the two values.

Arithmetic/Logical Instructions

- Logical operations can also operate on ***immediates***. These ***l-type*** instructions are ***andi***, ***ori***, and ***xori***.
- ***nori*** is not provided, because the same functionality can be easily implemented using the other instructions.
- **Figure 6.15** shows examples of the *andi*, *ori*, and *xori* instructions.
 - The figure gives the values of the source register and immediate and the value of the destination register **rt** after the instruction executes.
- Because these instructions operate on a 32-bit value from a register and a 16-bit immediate, they **first zero-extend** the immediate to 32 bits.

Arithmetic/Logical Instructions

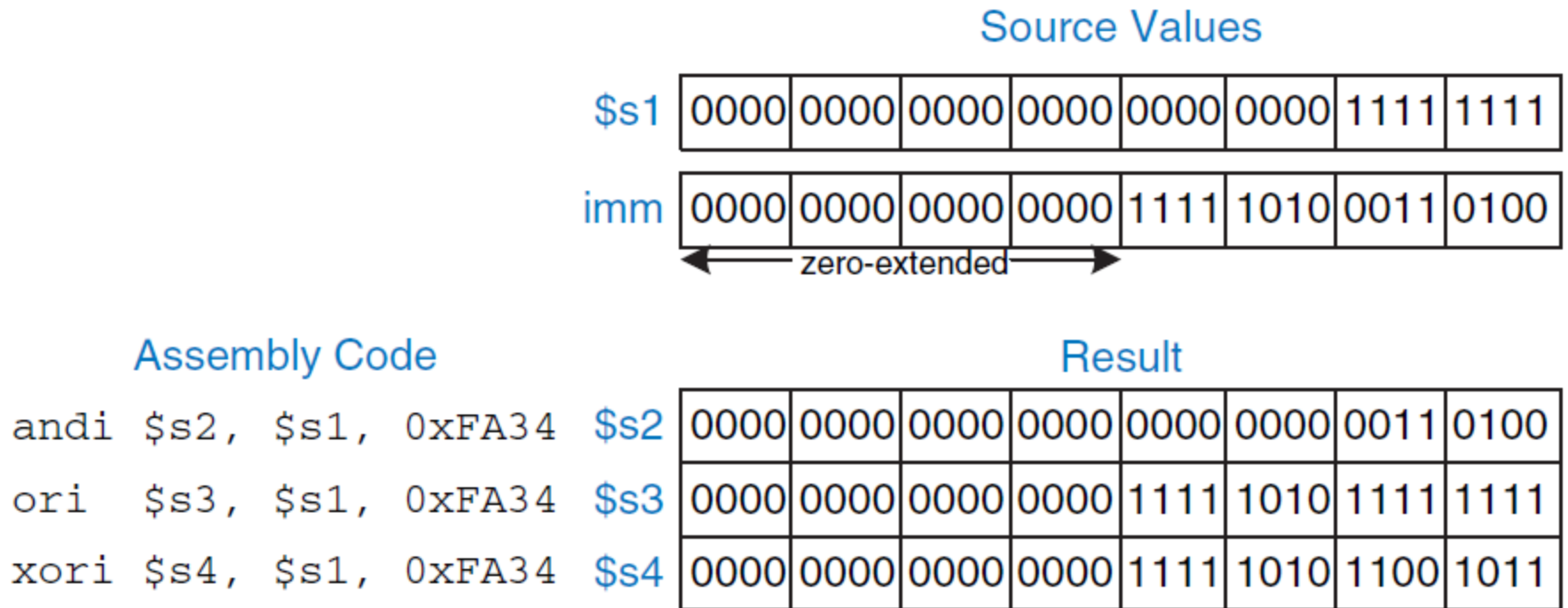


Figure 6.15 Logical operations with immediates

Shift Instructions

- Shift instructions shift the value in a register **left** or **right** by up to 31 bits. Shift operations multiply or divide by powers of two. MIPS shift operations are ***sll*** (shift left logical), ***srl*** (shift right logical), and ***sra*** (shift right arithmetic).

Shift Instructions

Assembly Code		Field Values					
		op	rs	rt	rd	shamt	funct
sll \$t0, \$s1, 4		0	0	17	8	4	0
srl \$s2, \$s1, 4		0	0	17	18	4	2
sra \$s3, \$s1, 4		0	0	17	19	4	3
		6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Machine Code						
op	rs	rt	rd	shamt	funct	
000000	00000	10001	01000	00100	000000	(0x00114100)
000000	00000	10001	10010	00100	000010	(0x00119102)
000000	00000	10001	10011	00100	000011	(0x00119903)
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	

Figure 6.16 Shift instruction machine code

Generating Constants

Code Example 6.10 16-BIT CONSTANT

High-Level Code

```
int a = 0x4f3c;
```

MIPS Assembly Code

```
# $s0 = a  
addi $s0, $0, 0x4f3c    # a = 0x4f3c
```

Code Example 6.11 32-BIT CONSTANT

High-Level Code

```
int a = 0x6d5e4f3c;
```

MIPS Assembly Code

```
# $s0 = a  
lui $s0, 0x6d5e        # a = 0x6d5e0000  
ori $s0, $s0, 0x4f3c    # a = 0x6d5e4f3c
```

Multiplication and Division Instructions

- **Multiplication** and **division** are somewhat different from other arithmetic operations.
- **Multiplying** two 32-bit numbers produces a **64-bit product**.
- **Dividing** two 32-bit numbers produces a **32-bit quotient** and a **32-bit remainder**.
 - The MIPS architecture has **two special-purpose registers**, *hi* and *lo*, which are used to hold the **results of multiplication and division**.
 - **Mult \$s0, \$s1** multiplies the values in **\$s0** and **\$s1**.
 - The **32 most significant bits** of the product are placed in *hi* and the **32 least significant bits** are placed in *lo*.
 - Similarly, **div \$s0, \$s1** computes **\$s0/\$s1**. The **quotient** is placed in *lo* and the **remainder** is placed in *hi*.

Branching

- A computer performs different tasks depending on the input. For example, **if/else** statements, **switch/case** statements, **while loops**, and **for loops** all *conditionally execute code depending on some test*.
- To sequentially execute instructions, the **program counter (PC) increments by 4 after each instruction**.
- **Branch instructions** modify the program counter to skip over sections of code or to repeat previous code.
- **Conditional branch** instructions perform a test and branch only if the test is TRUE.
- **Unconditional branch** instructions, called ***jumps***, always branch.